ordered some products by joining the orders table with itself on the basis of *employee_id* and *customer_id:*

```
use northwind;
select
a.id,a.employee_id,b.customer_id,b.id
from orders as a, orders as b
where a.employee_id = b.customer_id
group by a.employee_id
```

The analysis report of the self-join query is as follows:

ID	employee_id	customer_id	ID
41	1	1	44
31	3	3	36
32	4	4	31
33	6	6	37
46	7	7	41
37	8	8	33
30	9	9	46

The same idea can be used to analyse the table orders to find out those employees who have ordered on the same date.

```
use northwind;
select
a.id,a.employee_id,b.customer_id,b.
id,a.order_date,b.order_date
from orders as a, orders as b
where a.order_date = b.order_date and
a.employee_id = b.customer_id
group by a.employee_id
```

The analysis report of the above query is as follows:

ID	employee_id	customer_id	ID	order_date	order_date
41	1	1	44	3/24/2006 0:00	3/24/2006 0:00
59	4	4	58	4/22/2006 0:00	4/22/2006 0:00
47	6	6	47	4/8/2006 0:00	4/8/2006 0:00
50	9	9	46	4/5/2006 0:00	4/5/2006 0:00

In a similar case, if we consider the world database, then to find all the cities in India for which the name of the city and the district are the same, one can try the following query:

```
use world;
select distinct
a.ID,a.Name,b.District,c.Name
from city as a, city as b
inner join country as c on c.Code=b.
CountryCode
where a.Name = b.District and
c.Name='India'
```

The analysis report shows the following:

ID	Name	District	Name
1025	Delhi	Delhi	India
1073	Chandigarh	Chandigarh	India
1150	Pondicherry	Pondicherry	India

Union

From the analysis point of view, the union of tables is a useful feature of SQL. It provides a combination of more than one table on the basis of common field values. This set union functionality has several uses. One of the most practical uses may be analysing the current transaction table with archive tables. Here, to illustrate the use of union, I have considered the overall frequency distribution of job categories among the employees and customers in the Northwind database.

```
select a.job_title,count(a.job_title)
Frequency
from (select * from customers
union
select * from employees) as a
group by a.job_title
order by a.job_title
```

Intersection

Intersect is used to find common records from the participating tables. For instance, to find all the common records from both the customer and employee tables, one can use the following code:

```
select concat_ws(" ",c.first_name,c.
last_name) as name, c.city
from
     (select a.city,a.last_name,a.first_
name from customers as a
     intersect
     select b.city,b.last_name,b.first_
name  from employees as b) as c
```

Subquery

Subquery is an essential tool for data analysis. It provides a virtual table during execution of the existing query. In the above union and intersect queries, the subquery feature is used to provide union and intersection of customers and employees to the main query.

Though SQL supports stored queries like VIEW and STORED PROCEDURE, it suffers from a lack of flexibility. As it is embedded within an RDBMS, it cannot exhibit all the flexibility of a programming language and is also not safe. In addition to this, since its data visualisation capacity is limited, very often it becomes essential to port data to another platform for further analysis and visualisation. R and Python are good for data analysis, while SQL is commonly used as an embedded program from these two languages. This gives additional programming capabilities to SQL, and makes the data analysis and visualisation more versatile. END

By: Dipankar Ray

The author is a member of IEEE, IET (UK) and has more than 20 years of experience in open source versions of UNIX operating systems.